

🧠 Step 1 — Define your MVP (VERY IMPORTANT)

Don't start with full "research paper system".

Your first version should do ONLY this:

User query → metadata filter → keyword search → ranked results → explanation

That's it.

No vectors yet. No LLM yet.

📁 Step 2 — Setup project structure

Create this:

```
neuro-symbolic-rag/  
|  
├─ app.py      (FastAPI entry)  
├─ db.py       (DB connection)  
├─ models.py   (schema)  
├─ rules.py    (symbolic rules engine)  
├─ search.py   (BM25 / TF-IDF)  
├─ explain.py  (why this result came)  
├─ data/  
|   └─ docs.json
```

📄 Step 3 — Create dataset (start small)

Make 20–50 documents.

Example:

```
{  
  "id": 1,  
  "title": "AI Regulation EU 2025",  
  "content": "EU law on AI systems...",  
  "domain": "legal",  
  "verified": true,  
  "date": "2025-03-10"  
}
```

Keep it simple.

🌸 Step 4 — Build Symbolic Rule Engine (core idea)

This is the “neurosymbolic” part (first half).

```
def apply_rules(query):
    filters = {}

    if "legal" in query:
        filters["domain"] = "legal"

    if "verified" in query:
        filters["verified"] = True

    if "last 6 months" in query:
        filters["date_range"] = "6m"

    return filters
```

👉 Step 5 — Store & filter data (SQL or in-memory first)

Start EASY (Python list), not DB yet.

```
def filter_docs(docs, filters):
    results = docs

    if "domain" in filters:
        results = [d for d in results if d["domain"] == filters["domain"]]

    if "verified" in filters:
        results = [d for d in results if d["verified"] == True]

    return results
```

🔍 Step 6 — Add search (BM25 or TF-IDF)

Install:

```
pip install rank-bm25
```

Basic search:

```
from rank_bm25 import BM25Okapi

def bm25_search(docs, query):
    corpus = [doc["content"].split() for doc in docs]
    bm25 = BM25Okapi(corpus)

    scores = bm25.get_scores(query.split())
    ranked = sorted(zip(docs, scores), key=lambda x: x[1], reverse=True)

    return [r[0] for r in ranked]
```

Step 7 — Add explanation layer (THIS makes it “standout”)

```
def explain(doc, filters):
    reasons = []

    if doc["domain"] == filters.get("domain"):
        reasons.append("Matched domain rule")

    if doc["verified"]:
        reasons.append("Verified source")

    return reasons
```

Output:

```
{
  "title": "...",
  "reasons": [
    "Matched domain rule",
    "Verified source"
  ]
}
```

Step 8 — FastAPI API (final glue)

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/search")
```

```
def search(q: str):  
    filters = apply_rules(q)  
    docs = load_docs()  
  
    filtered = filter_docs(docs, filters)  
    ranked = bm25_search(filtered, q)  
  
    return ranked
```

Run:

```
uvicorn app:app --reload
```

🌟 What you have now (already impressive)

Even WITHOUT AI:

- Rule-based reasoning ✓
- Structured filtering ✓
- Retrieval system ✓
- Ranking system ✓
- Explainability ✓

This alone = **strong AI engineering foundation**